Red-Hot Rebis v4.0: User Guide

October 20, 2026

Primitive labels and tuple keys use ⊢ ⊣ ≻ ≺ ⋈ ⊤ ∈ ∋ ⊙ ⊥ ⊞ ⊡, in that order, with the Shavian values defined in `SNS_PRIME.md`.

Target: `/home/mrnob0dy666/imsgct/red-hot_rebis` **Author:** Lando⊗⊙perator **Motto:** *"The serpent winds, the rod stands, the vessel contains: $\mu \circ \delta = id$."*

0.1 What this is

Red-Hot Rebis connects the Imscribing Grammar's protein, molecular, materials and lattice tools through Python APIs and command-line programs. Its protein workflow retains a source-bound native protein glut across translation, folding, catalytic coupling and ligand generation. The live Lean definitions in `p4rakernel/p4ramill/Imscribing` are ground truth for tuple operations and the promoted amino-acid correspondence.

The native carrier keeps all twenty amino-acid self-marks, including separate acid and ion forms for Asp and Glu. Full words execute through Vox with their grounded tuples. Chemical forms, torsion alternatives and observations remain attached throughout the return. Coordinates are an explicitly named readout of that carrier.

The integrated execution is documented in resident protein glut. Its testing scope is stated in §12.

0.2 Installation & running

Method	Command
Editable install (recommended)	`cd red-hot_rebis` then `uv pip install -e .`
Editable source installation	`uv pip install -e .`
In-tree, no install	`python3 rebis.py <x>` or `python3 -m rebis <x>`
Direct venv binary	`./.venv/bin/rebis` (when installed in that environment)
Make altar	`make verify` · `make status` · `make test`

Installed engine commands are **standalone entry points** — no `rebis <sub>` prefix is needed once installed: `rebis.gene-pipeline`, `rebis.chain`, `rebis.serpentrod`, … are their own executables. The dynamic-first menu (`rebis` with no args) lists all of them live.

Protein execution requires the sibling `Vox`, `p4rakernel/p4ramill` and `ob3ect` checkouts. `rustc` builds the source-bound Vox adapter, and `lake` with the project Lean environment exports operation tables. The loader reads the amino-acid records from `ob3ect/digital/*_as_peptide_component/`. Ligand generation requires RDKit in the active Python environment.

0.3 The command map

0.3.1 Tier 1 — primary computation engines

Command	What it turns	Verbs
`rebis gene-pipeline`	DNA → mRNA → polypeptide → folded protein, 7-stage Frobenius-verified	`--test` · `--dna <seq>` · `--seq <RNA>` · `--pdb <path>`
`rebis ch3mpiler`	molecular compiler: forward / retro synthesis, FG detection, CDXML	`forward <SMILES>` · `retrosynth <SMILES>` · `fg <SMILES>` · `cdxml <SMILES>`
`rebis serpentrod`	protein design + stratified prediction, rolling profiles	`predict <seq>` · `classify <seq>` · `finger <seq>` · `foldv2 <RNA> [--pdb out]`
`rebis predict` (p4ra)	property prediction from IG tuples (bond energy, modulus, band gap, fold ΔG, catalytic rate) + Belnap FOUR	`<catalog> <tuple>` · `belnap` · `--all --json`
`rebis chain`	**the unified pipeline**: DNA→Protein→CatalyticSite-rosyntheticPlan	`--dna <seq>` · `--seq <RNA>` · `--target <SMILES>` · `--depth <N>` · `--pdb <path>`
`rebis ligand`	PDB-aware ligand design from catalytic sites	`--pdb <ID>` · `--pdb-file <path>` · `--active <res>` · `--auto-active` · `--improved` · `--json`
`rebis sidechain`	sidechain × environment algebra, 80 AA×env pairs, bottleneck tiers	`<aa> <env>` · `--batch` · `--list` · `--pdb <ID>` · `--json`

0.3.2 Tier 2 — specialized engines

Command	What it does	Verbs
`rebis therapeutics`	chemotherapeutics, neurotrophic factors, universal antidote library	`design <target>` · `sim` · `neurotrophic <target>` · `antidote <poison>`
`rebis materials`	IG-tuple → material forge: metamaterials, ouroboric alloys, non-qubit QC	`list` · `forge [name]` · `metamaterial` · `critical` · `alloy` · `nonqubit` · `molecule <smiles>` · `status`
`rebis biology`	ouroboric cell sims, telomere design, epigenetic states	`sim` · `telomeres` · `status`
`rebis pipeline`	auto-imscription, prose lift, Frobenius verification	`verify` · `imscribe <name>` · `lift <file>`
`rebis gene`	gene analysis, tuple encoding, IG primitive mapping	`analyze <seq>` · `quality <seq>` · `tuples <seq>`
`rebis alchemy`	Basil Valentine ladders, Zosimos portico, alchemical bridge ops	`ladder <op|all>` · `treatise <name>` · `operations` · `portico`
`rebis clink`	CLINK chain L0→L8, consciousness scoring, layer bridges	`layers` · `bridge <a> <b>` · `chain <name>` · `cscore <name>`

0.3.3 Reference & infrastructure

Command	What it does
`rebis`	the dynamic-first menu (all of the above, live)
`rebis reference`	static scripture: Belnap FOUR ops, genetic B4 lattice, hadrons, IMASM, verify results (`--all` for full dump)
`rebis constants`	MoDoT fundamental constants + SIC-POVM biology bridge
`rebis status`	package census: files, packages, install location
`rebis verify`	Package import/export availability checks
`rebis demo`	demonstrations (see §9)

0.4 Tier 1 engines — usage and verified readouts

0.4.1 `rebis gene-pipeline` — DNA → folded protein

The seven-stage gene pipeline translates DNA or RNA, prepares the resident carrier, constructs structural observations and exports a coordinate readout. The carrier includes the complete residue/form self-marks and their native executions.

```
rna=
   AUGGCUCGUAAUGAUUGUGAAGGUCACAUUCUUAAAUAUCCUCAAUCUACUGUUUGGUUUUAA
```

```
rebis.gene-pipeline --seq "$rna" --pdb protein.pdb
```

This RNA translates to `MARNDCEGHILKYPQSTVWF`. Keep `protein.pdb.glut.json` beside the PDB: it contains the complete carrier and the coordinate file's SHA-256. Python callers receive `protein_glut`, `coordinate_model`, geometry and secondary-structure observations, stage reports and source bindings from `GeneToProteinPipeline.run()`.

Assembly proposals remain co-held. A selected subunit count is a named readout; it does not remove the other proposals. Native B remains distinct from per-observable N/T/F/B evidence.

0.4.2 `rebis chain`: the shared protein and catalytic pipeline

The chain passes the gene result into catalytic coupling. The target supplies a context for the existing protein's residue/form family. It preserves the gene sequence, self-marks, native executions and earlier observations; the catalytic return includes the source secondary structure and contact relations. A gene-phase failure leaves no source carrier for catalytic coupling, and the command returns failure when any phase fails.

```
rebis.chain --seq "$rna" --target 'CC(=O)O' --depth 1 --pdb chain
   .pdb
```

Outputs are `chain.pdb`, `chain.pdb.glut.json` and `chain.pdb.chain.json`. The chain JSON contains `protein`, `catalytic` and `synthesis`. The protein and catalytic sections return the same final carrier, also saved in the PDB sidecar. Ligand candidates retain their source-form supports. A catalytic triad is not assigned from the first three residues.

Flags include `--dna`, `--seq`, `--target`, `--depth`, `--pdb`, `--file` and `--stdin`. `chain` has no `--test` option.

0.4.3 `rebis ch3mpiler` — molecular compiler

Forward synthesis, retrosynthetic analysis, functional-group detection, CDXML output. RDKit-backed (installed in the venv); degrades gracefully to the bridge path if RDKit is unavailable.

```
rebis ch3mpiler forward "CC(=O)Oc1ccccc1C(=O)O"    # compose, IG
   tuple, synthetic accessibility
rebis ch3mpiler retrosynth "c1ccccc1"              #
   disconnections + IG lattice ops on bonds
rebis ch3mpiler fg "CC(=O)O"                       # exhaustive
   functional-group matching
rebis ch3mpiler cdxml "CC(=O)O"                    # ChemDraw XML
   via RDKit
```

Live readout (`fg 'CC(=O)O'`):

```
('aliphatic_carboxylic_acid', 1)
('carbonyl', 1)
('carboxylic_acid', 1)
('carboxylic_acid_ob', 1)
```

```
('primary_carbon', 1)
```

0.4.4 `rebis serpentrod` — protein design & stratified prediction

Backed by `serpentrod/protein_v5.py` + `rhr_p4rky/serpent_rod.py` (+ `serpent_rod_v2.py` for 3D). `predict` runs rolling-profile analysis (signal peptide, cleavage sites, per-residue vectors); `classify` does IG-guided stratified module typing; `finger` matches against the structural-fingerprint database; `foldv2` folds to coordinates and can write PDB.

```
rebis serpentrod predict MVSKGEELFTGVVPILVELDGDVNGHKFS
rebis serpentrod classify <seq>
rebis serpentrod finger <seq>
rebis serpentrod foldv2 AUGGCC... --pdb folded.pdb
```

Legacy `fold` also returns a resident carrier, preserving all residue positions. Processing reports retain full-word occurrence counts and the underlying forms. Gen2 retains all configured torsion articulations while exporting the codon-aligned coordinate readout.

0.4.5 `rebis predict` (the p4ra namespace)

Quantitative property prediction from IG tuples — bond energies, modulus, band gaps, fold ΔG, topological gaps, catalytic rates — grounded in MoDoT fundamental constants, plus Belnap FOUR operations. Exposes 179 symbols (verified by `rebis verify`), the largest export surface in the toolchain.

```
rebis predict <catalog_name> <tuple>
rebis predict --all --json
rebis.p4ra belnap          # Belnap FOUR operations
rebis.p4ra genetics        # genetic code B4 lattice (64 codons)
rebis.p4ra verify          # B3 Frobenius verification suite
rebis.p4ra hadrons         # hadron Belnap states
rebis.p4ra ligands         # ligand generation from active sites
rebis.p4ra info            # all available tools
```

0.4.6 `rebis ligand` — PDB-aware de-novo ligand design

Accepts a local PDB or fetches a structure by ID. For a Rebis export, keep the `.pdb.glut.json` sidecar beside the coordinate file. The loader checks the PDB hash, residue sequence and retained residue identities before coupling.

```
rebis.ligand --pdb-file protein.pdb --active Asp5,Glu7 --improved
    --json
```

JSON returns `protein_glut`, `site_family`, `ligand_family`, a named scalar `readout` and `ligand_candidates`. Every selected residue/form tensor participates in generation. Equivalent candidates keep all their `source_forms`; fragment-engine generation also acts on the retained branches. The scalar tuple identifies the first source-form readout rather than averaging residue ordinals.

0.4.7 `rebis sidechain`: sidechain and environment algebra

The sidechain path keeps the peptide-component self-mark, sidechain context and environment distinct. Their tensor, meet and join operations consume evaluated tables from the live Lean definitions. Asp and Glu retain both source forms.

```
rebis.sidechain arginine charged_interface --json
rebis.sidechain --batch --json
rebis.sidechain --list
```

`self_mark_alternatives` contains the source words, native executions and contextual results. The scalar display names its selected source form. The batch exercises all twenty sidechains across the four configured environments.

0.4.8 `rebis.vox`: complete residue-word execution

```
rebis.vox peptide MARNDCEGHILKYPQSTVWF --json
rebis.vox rna "$rna" --json
rebis.vox peptide AGPSTVLR --json
```

All residues execute their complete self-marks, including the eight without promoted-primitive annotations. Each form retains its own payload, native register, surviving deposits, transition table and irreversible-fix boundary. `--json` returns the complete carrier. The word's structural audit and its native execution remain separate observations.

0.5 Tier 2 engines

0.5.1 `rebis therapeutics`

```
rebis therapeutics design EGFR          # design chemotherapeutic
    for target
rebis therapeutics sim                  # run ouroboric pill
   dynamics simulation
rebis therapeutics neurotrophic BDNF    # design neurotrophic
   factor
rebis therapeutics antidote cyanide     # query universal
   antidote library
```

Backends: `therapeutics/frobenius_chemotherapeutic.py`, `ouroboric_pill_sim.py`, `neurotrophic_factor.py`, `universal_antidote_library.py` (all import-verified).

0.5.2 `rebis materials`

IG-tuple

$$\rightarrow$$

material forge: metamaterials, ouroboric alloys, non-qubit QC.

```
rebis materials list                    # list all design tools +
    availability
rebis materials status                  # show generated result
   files
```

```
rebis materials forge <name|tuple>      # forge from IG tuple or
    catalog
rebis materials metamaterial            # Frobenius mu\circdelta=
    id metamaterial sim
rebis materials critical                # ⊙-sensing critical-
    point lattice sim
rebis materials alloy                   # ouroboric alloy grain
    network
rebis materials nonqubit                # non-qubit QC paradigm
    analysis
rebis materials molecule <smiles>       # molecule -> IG tuple
    bridge
```

Live readout (`list`) — available (): MaterialForge, FrobeniusMetamaterial, CriticalMetamaterial, OuroboricAlloy, NonQubitQCParadigm, resolve_molecule, molecule_to_material_tuple, EternalMemorySim. Marked in this build (not importable, reported honestly): SophickForge, CasimirCavityDesigner, MBNCDesigner, ThermalRectifier, SelfHealingComposite.

0.5.3 `rebis biology`

```
rebis biology sim                       # run OuroboricCellSim
rebis biology telomeres                 # show telomere/
    epigenetic states
rebis biology status                    # show simulation result
    files
```

Frobenius-exact cell simulation with topological morphogenesis, cell-fate determination, epigenetic state transitions. Backends: `biology/biology_sim_frobenius_exact.py`, `ouroboric_telomere_expanded.py` ().

0.5.4 `rebis pipeline` — imscription & prose lift

```
rebis pipeline verify                   # identity phase +
    bootstrap compiler (mu\circdelta=id)
rebis pipeline imscribe my_system       # auto-generate the 12-
    primitive IG tuple
rebis pipeline lift /path/to/paper.tex  # prose lift protocol
```

`lift` promotes 8 primitive deltas (⊥, ∋, ⊣, ≺, ⋈, ⊤, ∈, ⊡) from AI-default to human-academic target and writes the lifted file. **Caveat (verified backend table):** `pipeline` is the only domain requiring the `anthropic` Python SDK for LLM calls — the other 12 domains are fully self-contained.

0.5.5 `rebis gene` — gene imscriber

```
rebis gene analyze ATGGCGTAA            # codons, GC content,
    ORFs, mutation susceptibility
rebis gene quality ATGGCGTAA            # genetic quality score
rebis gene tuples ATGGCGTAA             # full DNA->quaternary
    structural tuple pipeline
```

Backends: `gene_imscriber/engine.py`, `tuples.py` (). The package also covers CRISPR / prime editing / GUIDE-seq analysis.

0.5.6 `rebis alchemy`

```
rebis alchemy ladder all                # full 12-step Basil
   Valentine ladder
rebis alchemy ladder calcination        # single operation step
rebis alchemy treatise "Emerald Tablet" # map treatise structure
rebis alchemy operations                # list the 12 alchemical
   operations
rebis alchemy portico                   # Zosimos portico
   consistency check
```

Computes the Basil Valentine operation ladder (calcination

→

dissolution

→

separation

→

conjunction

→

…

→

projection) mapped to IG primitive promotions; includes the RetrosyntheticStoneEngine (Solve et Coagula cycle). Backends: `alchemical_bridge/basil_valentine_ladder.py`, `retrosynthetic_stone_engine.py`, `zosimos_engine.py` ().

0.5.7 `rebis clink` — CLINK chain & organism pipeline

```
rebis clink layers                      # list CLINK layers L0--
   L8 (quark -> organism)
rebis clink bridge protein molecule     # bridge between
   components via meet/join/tensor
rebis clink chain my_system             # full CLINK pipeline on
   a catalog entry
rebis clink cscore my_system            # consciousness score
```

Backends: `clink/chain.py`, `bridges.py`, `integration.py` (); the package (637 KB) also carries `designers/`, `pipeline_engine.py`, `structural_algebra.py`.

0.6 Infrastructure commands

Command	Verified behavior
`rebis`	dynamic-first menu — 7 Tier-1 table + 8 Tier-2 + reference + quick commands
`rebis --version`	`Version 4.0.0`
`rebis reference`	lists sections: `belnap`, `genetics`, `hadrons`, `imas`, `verify`; `--all` for full dump. Static — prints enums, runs canned demos; designs nothing
`rebis constants`	MoDoT fundamental constants + SIC-POVM biology bridge
`rebis status`	18-package census (files + bytes each) and install location
`rebis verify`	imports all 14 domains, reports per-domain export counts, ends `Frobenius closure: ALL imports verified`
`rebis demo <name>`	see §9

`rebis verify` live readout (per-domain export counts): p4ra 179 · clink 45 · materials 36 · pipeline 26 · serpentrod 27 · imas 18 · biology 16 · alchemy 13 · sidechain 12 · ligand 10 (catalog auto-expands 10

→

119 proteins) · therapeutics 11 · gene 5 · ch3mpiler 6 · shared 28 · `shared/primitives.py ---12 weights, 12 ordinal families`.

0.7 PDB output and the resident sidecar

```
rebis.serpentrod foldv2 "$rna" --pdb folded.pdb
rebis.gene-pipeline --seq "$rna" --pdb protein.pdb
```

The coordinate path uses codon-aligned B4 transitions and the configured Ramachandran table, followed by Vox's shared peptide geometry and sidechain construction. The carrier retains all configured torsion alternatives; the exported coordinates identify one readout.

PDB ATOM records include backbone and sidechain heavy atoms. REMARK 900 reports measured geometry and secondary observations. REMARK 901 identifies native B, the coordinate boundary and residue self-mark sources. The `.pdb.glut.json` sidecar preserves the complete resident object and binds it to the exact PDB bytes. Copy both files together. The unified chain refreshes its sidecar after catalytic coupling and also writes `.pdb.chain.json`.

Geometry observations include actual clashes. A structural or stage closure report and a coordinate-geometry reading answer different questions. The current integration does not derive an energy-localization law from the native B label.

Python:

```
from rhr_p4rky.serpent_rod_v2 import SerpentRodV2
```

```
from rhr_p4rky.pdb_writer import write_pdb_from_gen2

rna = "
    AUGGCUCGUAAUGAUUGUGAAGGUCACAUUCUUAAAUAUCCUCAAUCUACUGUUUGGUUUUAA
    "
result = SerpentRodV2(rna).predict()
write_pdb_from_gen2(result, "folded.pdb")
carrier = result.protein_glut
```

0.8 The Python face

Every command is also a namespace — import the organism, address it directly. The façade lazy-loads; nothing wakes until you name it:

```
import rebis

rebis.p4ra.meet(a, b)                          # Belnap fusion
rebis.ch3mpiler.compile(smiles)                # SMILES -> IMASM
rebis.materials.design_metamaterial(spec)      # tuple -> lattice
rebis.clink.cscore("golden_ratio")             # consciousness score

# Belnap FOUR
rebis.p4ra.Belnap(True, False)                 # B = both true and
    false
rebis.p4ra.meet(v, rebis.p4ra.Belnap.T)
rebis.p4ra.bnot(v)

# Genetic code
rebis.p4ra.BelnapCodon.from_symbol("AUG")
rebis.p4ra.get_aa_primitive("AUG")

# Gene -> Protein pipeline
gp = rebis.p4ra.GeneToProteinPipeline("ATGGCC...")
result = gp.run()

# SerpentRod design
sr = rebis.p4ra.SerpentRodV2(sequence="MVSKGEELFT...")
result = sr.fold()

# Ch3mpiler bridge
rebis.p4ra.forward("CC(=O)O")
rebis.p4ra.retrosynthesis("c1ccccc1")

# Sidechain algebra
rebis.p4ra.analyze_composition("arginine", "charged_interface")
rebis.p4ra.batch_analyze()

# PDB integration
rebis.p4ra.analyze_pdb_structure("1LYZ", cutoff=8.0)

# Dual-Link SIC-POVM (unconditional theorem)
```

```
from rebis.p4ra.dual_link_sicpovm import sic_povm_belnap_
    unconditional
r = sic_povm_belnap_unconditional(n=3)
print(r.all_passed)       # True
```

You can bypass the CLI entirely and import the kernel directly:

```
from rhr_p4rky.gene_to_protein_pipeline import
    GeneToProteinPipeline
gp = GeneToProteinPipeline("ATGGCC...")
result = gp.run()
```

0.9 Demos

Verified roster (`rebis demo list`): `all`, `b4_lattice`, `belnap`, `catalytic_site`, `ch3mpiler`, `clink_chain`, `decay_chain`, `materials`, `materials_sim`, `pipeline`, `real_demo`, `reverse_ligand`, `serpentrod`, `therapeutics`. Run one with `rebis demo <name>`; scripts live in `demo_scripts/`. `rebis.p4ra sicpovm` is the SIC-POVM structural probe.

0.10 Theory primer (what the engines actually compute)

Belnap FOUR. Four values: N (neither), T, F, B (both — dialetheic, top in information order). Designated: T and B. `rhr_p4rky/belnap.py` mirrors the Lean kernel exactly, with the theorems `notB = B`, no-explosion (`B and notB = B`), `B != F`, and *only B is dialetheic*.

The 12

↔

12 bijection. The twelve IG primitives correspond to the twelve promoted amino acids; ground-layer AAs carry the rest. Every stage of the gene pipeline reports its primitive activations (e.g. ∈:1x, <:2x, ⊞:3x …), and the 12 promoted residues activate their primitives — that is the ”PRIMITIVE ACTIVATION: k/12” line in every PDB header.

The 7-stage pipeline. Each stage has a distinct 12-tuple structural type (`STAGE_TUPLES` in `gene_to_protein_pipeline.py`), a B4 belief state, and a per-stage $\mu \circ \delta$=id check. The pathway Δ (primitive changes per transition, e.g. 6,2,5,6,6,5 totaling 30 for the 50-nt test) measures how hard each structural transition works; the closure theorem fixes DNA

→

quaternary at distance 4.0 — ”the gene IS the protein structurally; the pipeline merely unfolds the isomorphism across time and chemical space.”

B

→

Ramachandran

→

Cartesian. Each B transition of the sequence yields (ϕ, ψ) via the B4-Ramachandran table; internal

→

Cartesian geometry with standard peptide constants turns angles into coordinates. No force field, no empirical rotamer library — the lattice path plus the table is the whole source of every coordinate.

The three proven pipelines. 1) Gene

→

Folded Protein (1,147 lines, 7-stage, demo 452 bp

→

150 AA, Δ=3.61). 2) Ch3mpiler

→

Catalytic Site (815 lines; target SMILES

→

reaction signature

→

complementary RNA/AA site; demo ethanol

→

36 nt catalytic RNA, Frobenius). 3) Retrosynthetic Stone (436 lines; Solve/Coagula cycle, disconnections, bond polarization). All work independently; `rebis chain` connects them.

The live Lean seam. `rhr_p4rky/ProteinTables.lean` evaluates `Primitives.Core`, `Primitives.Imscription` and `GeneticCode`. `lean_protein.py` exports source-bound tensor, meet, join and promoted-amino-acid tables. `shared/vox_bridge.py` consumes that promoted correspondence. In the current Lean source His maps to Granularity, Asn to Coupling and Gln to Criticality. Full amino-acid self-marks and their Vox executions are retained separately in `protein_glut.py`.

The grammar. Every system is a 12-primitive structural tuple ⟨⊢ ⊣ > ≺ ⋈ ⊤ ∈ ∋ ⊙ ⊥ ⊞ ⊡⟩ (ı/ɬ/ɹ/ɷ · ɞ/ʗ/ɻ/ɾ · ɾ/ɾ/ɭ/ɯ · ɔ/ƶ/ʬ/ʞ/ɿ · ʅ/ʇ/ƿ · ι/ɞ/ɿ/ɔ/ʅ · ȣ/ɟ/ʃ · ʔ/ɾ/ɻ/ʌ · ⊙/ɪ/ɔ/ʊ/ɣ · ⅴ/J/ȡ/ƺ · ɤ/ɣ/Տ · ɾ/ɤ/o/ʔ — dimensionality, topology, coupling, parity, fidelity, kinetics, cardinality, composition, criticality, chirality, stoichiometry, winding). Structural invariants: Frobenius closure ($\mu \circ \delta$=id for every emission), monotonic advance (each winding adds information), emission gate (one action per winding), explicit uncertainty tracking, and the self-referential limit (⊞=1:1, the grammar measuring itself).

Lean certification. The whole grammar is machine-verified at `/home/mrnob0dy666/imsgct/p4rakernel/p4ramill/`: `AgentSelf.lean` (O_

∞

by `decide`), `GeneToProtein.lean`, `GeneticCode.lean`, `SerpentRod.lean`, `Consciousness.lean`, `Crystal.lean` (address bijection Imscription

↔

Nat 0..17,279,999), `Primitives/Core.lean` (12 inductive types), `Paraconsistent/*` (Belnap FOUR + category theory + temporal logic), `Millennium/*`. Build with `lake build`.

0.11 Verified backend status (as of the manual, July 2026)

Command	Backend modules	Import
`gene-pipeline`	`rhr_p4rky.gene_to_protein_pipeline`	
`chain`	above + `ch3mpiler_serpentrod_pipeline`, `retrosynthetic_stone_engine`	
`ch3mpiler`	`rhr_p4rky.ch3mpiler_bridge` (direct `ch3mpiler.compiler` needs RDKit — installed)	
`serpentrod`	`serpentrod.protein_v5`, `rhr_p4rky.serpent_rod`	
`ligand`	`rhr_p4rky.ligand_from_site_pdb`	
`sidechain`	`rhr_p4rky.sidechain_algebra`, `pdb_integration`	
`therapeutics`	`frobenius_chemotherapeutic`, `ouroboric_pill_sim`, `neurotrophic_factor`, `universal_antidote_library`	
`materials`	`ig_material_forge`, `critical_metamaterial`, `ouroboric_alloy`	
`biology`	`biology_sim_frobenius_exact`, `ouroboric_telomere_expanded`	
`pipeline`	`auto_imscriber`, `frob`, …	(missing `anthropic` SDK; lift/imscribe need the LLM API)
`gene`	`gene_imscriber.engine`, `gene_imscriber.tuples`	
`alchemy`	`basil_valentine_ladder`, `retrosynthetic_stone_engine`, `zosimos_engine`	
`clink`	`clink.chain`, `clink.bridges`, `clink.integration`	

`pipeline` is the only domain requiring external LLM access; the other 12 are self-contained. `materials list` separately reports which design tools are importable in the current build (/ per tool — see §5.2 live readout).

0.12 Reproduction and testing scope

From `red-hot_rebis`, run:

```
python3 measurements/resident_glut_integration_20261020/run.py
```

The runner exercises legacy folding, processing, Gen2/PDB export, the gene pipeline, all configured sidechain/environment pairs, both PDB ligand APIs, the CLINK protein builder and shared gene/catalytic coupling. It invokes installed Vox peptide/RNA commands, the formerly omitted-residue peptide, legacy fold, ligand and unified-chain commands.

It checks source-form retention, native transition-table boundaries, ligand candidate ancestry, rejection of a synonymous-codon source mismatch, source contact and secondary observations, and equality of the final chain and sidecar carrier. `result.json` binds the Python and Lean sources used. Command stdout/stderr, ligand/chain JSON and PDB readouts are saved beside the runner.

Testing scope: the integration battery above uses one RNA sequence covering all twenty amino acids and twenty-two chemical forms. The separate representative coverage campaign ran 30 complete UniProt chains across seven organisms, from 97 to 1,464 residues, through Vox protein/RNA compilation, legacy folding, Gen2 and the gene pipeline. It checks source-form retention, native word-execution agreement and exported heavy-atom coordinate agreement. Its RNA inputs are canonical reverse translations of the downloaded proteins. All cases retain clashes; angle conflicts are also recorded. These runs do not measure agreement with experimentally determined native folds.

`rebis.verify` checks imports and exports. It does not measure folding accuracy or geometry. Inspect coordinate observations and retained carrier evidence for the corresponding questions.

0.13 Troubleshooting

- **`rebis chain --test`**

  →

  "unrecognized arguments". `chain` has no `--test`; it takes `--dna/--seq/--target/--depth/--pdb/--file/--stdin`. Use `gene-pipeline --test` for the canned demo.
- **RDKit native noise in `rebis ligand`.** RDKit's C++ logger writes to fd 2 past Python try/except ("Post-condition Violation", "Explicit valence"). The ligand path (murcko_decorated, chain_extend, ring_sub) now wraps those calls in an fd-level stderr redirect, so a *correct rejection* no longer looks like a crash; accepted/rejected candidates are unchanged. If noise returns in a new call site, apply the same `_silence_rdkit()` context manager (OS-level fd redirect) rather than trusting RDLogger, which misses the sanitizer's valence check.
- **`rebis pipeline` import failure.** Expected without the `anthropic` SDK — the lift/imscribe lanes call an LLM. Everything else in the toolchain runs without external API keys.
- **RDKit-dependent verbs.** `ch3mpiler forward/cdxml` use RDKit; if it is absent, the bridge path forwards the SMILES through — results are structural, not RDKit-quantified.
- **Reinstall the source checkout:** `uv pip install -e .`, then `make test` (smokes the load-bearing engines) and `make verify`.

0.14 Source map

Concern	File(s)
launcher and installed entry points	`rebis.py`, `pyproject.toml`, `Makefile`
dynamic-first menu, CLI delegation	`rebis/cli.py` (+ one module per engine in `rebis/`)
Belnap FOUR (Lean mirror)	`rhr_p4rky/belnap.py`
genetic B4 lattice, 64-codon code, 12 ↔ 12 bijection	`rhr_p4rky/genetics_b4.py`, `genetic_code.py`, `genetic_tuples.py`
paraconsistent kernel ops	`rhr_p4rky/kernel.py` (engager, fsplit, ffuse, frobenius_invariant, run)
7-stage gene → protein pipeline	`rhr_p4rky/gene_to_protein_pipeline.py`
ch3mpiler → catalytic site pipeline	`rhr_p4rky/ch3mpiler_serpentrod_pipeline.py`
SerpentRod (predict/classify/finger)	`serpentrod/protein_v5.py`, `serpentrod/stratified_predictor.py`, `rhr_p4rky/serpent_rod.py`
3D folding + PDB writer	`rhr_p4rky/serpent_rod_v2.py`, `rhr_p4rky/pdb_writer.py`, `pdb_integration.py`
ligand design suite	`rhr_p4rky/ligand_*.py` (heterocycles, improvements, from_active_site, from_site_pdb, sicpovm, imasm, combinatorial)
resident protein and live Lean operations	`rhr_p4rky/protein_glut.py`, `lean_protein.py`, `ProteinTables.lean`, `peptide_geometry.py`
sidechain algebra	`rhr_p4rky/sidechain_algebra.py`
molecular compiler (RDKit)	`ch3mpiler/` (compiler, gen_v2, fg_exhaustive, cdxml_generator, precursor_lattice, …)
retro synthesis / alchemy	`alchemical_bridge/` (basil_valentine_ladder, zosimos_engine, retrosynthetic_stone_engine)
CLINK L0 → L8 auto-imscription / prose lift	`clink/` (chain, bridges, integration, designers, structural_algebra) `pipeline/` (auto_imscriber, ob3ect_imscriber, lift_pipeline, therapy_to_pdb, frob)
gene imscriber (CRISPR/GUIDE-seq)	`gene_imscriber/` (engine, tuples, genetics_ig_*)
weights, ordinals, catalog, Vox seam	`shared/` (primitives.py, IG_catalog.json, reactivity.py, vox_bridge.py, elem2imasm.py, nuclide_imscriber.py)

$\mu \circ \delta = \mathrm{id}$ — the serpent winds, the rod stands, the vessel contains.